Relatório da Etapa 3 — Construção e avaliação dos classificadores

Trabalho em Competição: Detecção de Intrusão em Redes · Inteligência Artificial · Profa. Cristiane Neri Nobre

Nov 12, 2026 · @Luisa

Sumário executivo

A **random forest** é o melhor modelo da Etapa 3. Ela teve F1-macro de **0,9846 ± 0,0031** na validação cruzada 5×2 sem o atributo `score_ids_legado` e de **0,9909 ± 0,0016** com ele. Foi o modelo de maior desempenho e o mais estável nas duas versões. O conjunto de teste não foi usado em nenhuma decisão.

- **O que mais rendeu:** ajustar o limiar de decisão fora do fold avaliado. Só isso levou a random forest de 0,9756 para 0,9815, sem mudar o modelo.
- **Pré-processamento que entrou:** a imputação de `service` e `flag` por uma random forest, que acerta 97% das categorias escondidas, contra 57% da moda. Ela entrou na random forest (+0,0019, 9 de 10 folds) e na MLP. A MLP também ganhou com o MissForest nos números (+0,0057, 9 de 10 folds).
- **Hiperparâmetros:** 300 árvores, `max_features` = 0,2 e `max_depth` = 30 levaram a random forest a 0,9846. A MLP foi a que mais ganhou com o ajuste: 0,9698 → 0,9750.
- **O que não funcionou:**
  - balanceamento, com 4 técnicas testadas;
  - filtro de variância quase zero;
  - devolver ao treino os pares contraditórios reetiquetados;
  - ensembles;
  - três famílias de atributos novos.

  Todos foram rejeitados pela regra de comparação fold a fold.
- **Decisão:** levar para a Etapa 4 a random forest com limiar 0,37 (sem o score) ou 0,47 (com o score). A escolha entre as duas versões depende de o `score_ids_legado` ser aceito como atributo legítimo.

Objetivo e escopo

A Etapa 3 treinou e comparou os três algoritmos do grupo: **árvore de decisão**, **random forest** e **rede neural MLP treinada por backpropagation**. Cada algoritmo foi cruzado com as técnicas de pré-processamento, e o melhor modelo foi escolhido só pela validação no

treino.

- **Base:** `treino_limpo.csv`, com 55.074 conexões e 3,30% de anomalias, depois da limpeza da Etapa 1. A limpeza removeu as duplicatas, os 1.702 pares contraditórios e os dois atributos com vazamento.
- **Ponto de partida:** o pipeline da Etapa 2:
  - mediana nos números e moda nos textos;
  - `log1p` nas 16 contagens;
  - `StandardScaler`;
  - one-hot com os serviços raros (menos de 50 linhas) agrupados;
  - sem balanceamento e sem PCA.
- **Fora do escopo:** o arquivo de submissão e a explicabilidade ficam para a Etapa 4.
- **Entregável:** o notebook `etapa3_classificadores.ipynb`, com os resultados, as métricas e a tabela comparativa. Ele reaproveita os 99 experimentos registrados em `resultados_experimentos.jsonl` e roda em cerca de 1 minuto. Com `RODAR_TUDO = True`, refaz tudo do zero em 2 a 3 horas.

Metodologia de avaliação

Todo experimento usa a mesma validação cruzada: 5 folds estratificados, repetidos 2 vezes (`RepeatedStratifiedKFold`, `random_state` = 42). Isso dá 10 medidas por experimento, nos mesmos folds de todo o grupo, o que permite comparar técnicas fold a fold.

1. **Tudo dentro do fold.** O que aprende com os dados é ajustado só nos 80% de treino de cada fold: imputação (inclusive as florestas de imputação), escala, codificação, balanceamento e o modelo. Os 20% de validação só são transformados. Isso evita o vazamento descrito por Kaufman et al. (2012) e Arp et al. (2022).
2. **Métrica principal: F1-macro**, a mesma da competição. A acurácia não serve aqui: com 96,7% de conexões normais, um modelo que nunca acusa nada já tem 0,967 de acurácia.
3. **Limiar fora do fold.** O F1-macro depende do corte da probabilidade. Para cada fold, testamos os limiares de 0,02 a 0,98 nos **outros 4 folds da mesma repetição** e ficamos com o de maior F1-macro. Só então medimos o fold avaliado. Também mostramos o F1 com o limiar padrão de 0,5.
4. **Regra para aceitar uma técnica.** A técnica precisa ter ganho médio positivo **e** atender a uma de duas condições: vencer em pelo menos 8 dos 10 folds (a regra da Etapa 2) ou ter $p < 0{,}10$ no teste t corrigido de Bouckaert & Frank (2004). O teste t comum dá

"diferença significativa" demais, porque os folds compartilham linhas de treino (Nadeau & Bengio, 2003). A correção aumenta a variância:

$$t = \frac{\bar{d}}{\sqrt{\left(\frac{1}{k} + \frac{n_{val}}{n_{treino}}\right)\hat{\sigma}^2_d}}, \qquad k = 10,\ \frac{n_{val}}{n_{treino}} = 0{,}25,\ 9 \text{ graus de liberdade}$$

Aqui $\bar{d}$ é a diferença média de F1 entre a técnica e a referência nos 10 folds, e $\hat{\sigma}^2$ é a variância dessas diferenças. As técnicas são testadas **uma por vez**, como recomendam Kuhn & Johnson (2019): uma técnica aceita entra no pipeline, e a próxima é testada em cima dela.

Linhas de base

Com o pipeline da Etapa 2 sem nenhuma mudança, a random forest já sai na frente. **Só ajustar o limiar fora do fold leva a random forest de 0,9756 para 0,9815.**

Modelo	F1-macro, limiar 0,5	F1-macro, limiar ajustado	Desvio	Limiar mediano
Random forest (100 árvores)	0,9756	**0,9815**	0,0034	0,34
Árvore de decisão	0,9664	0,9664	0,0038	0,51
MLP (64 neurônios, 1 semente)	0,9636	0,9629	0,0066	0,45
Classe majoritária (sempre `normal`)	0,4916	—	0,0000	—

- **A acurácia engana.** A classe majoritária tem acurácia de 0,967 e F1-macro de 0,49, porque nunca encontra uma anomalia.
- **O limiar ótimo fica abaixo de 0,5.** É o que Lipton et al. (2014) mostram para o F1: num classificador calibrado, o melhor corte é metade do F1 máximo, e portanto nunca passa de 0,5.
- **Na árvore, o limiar não muda nada.** Sem poda, as folhas são quase todas puras, e a probabilidade é quase sempre 0 ou 1.
- **A MLP sai atrás.** O pipeline da Etapa 2 foi escolhido olhando a random forest e a regressão logística, não uma rede neural. Além disso, com uma semente só, a MLP varia ± 0,0066 entre os folds, o dobro da random forest.

Pré-processamento por algoritmo

Das 11 técnicas testadas, **só três aplicações passaram na regra**: a imputação de `service` / `flag` por floresta na random forest e na MLP, e o MissForest na MLP. Cada célula da tabela mostra o ganho de F1-macro sobre o passo anterior e em quantos dos 10 folds a técnica venceu. Em negrito, as técnicas aceitas.

Técnica	Árvore	Random forest	MLP, 1 semente	MLP, média de 3 sementes
Referência (pipeline da Etapa 2)	0,9664	0,9815	0,9629	0,9623
`service` / `flag` imputados por random forest	+0,0027 (7/10)	**+0,0019 (9/10), p = 0,07**	+0,0014 (6/10)	**+0,0018 (8/10)**
Números: MissForest	+0,0003 (5/10)	−0,0021 (1/10)	+0,0027 (6/10)	**+0,0057 (9/10), p = 0,06**
Números: mediana por `service`	−0,0010 (3/10)	+0,0004 (5/10)	−0,0012 (6/10)	—
Números: recalcular pela definição	+0,0019 (5/10)	−0,0014 (3/10)	+0,0008 (4/10)	—
Atributo novo: bytes = 0	−0,0003 (4/10)	−0,0004 (4/10)	−0,0041 (1/10)	—
Atributo novo: razões	−0,0012 (2/10)	−0,0017 (3/10)	−0,0006 (5/10)	—
Atributo novo: taxas de erro totais	+0,0001 (4/10)	−0,0013 (2/10)	−0,0041 (3/10)	—
Filtro de variância quase zero	−0,0051 (1/10)	−0,0094 (0/10)	−0,0139 (0/10)	—
Corte nos percentis 0,5–99,5	—	—	+0,0004 (5/10)	−0,0007 (5/10)
Oversampling aleatório	−0,0104 (0/10)	—	+0,0026 (5/10)	+0,0032 (6/10)
Pares contraditórios reetiquetados	−0,0016 (4/10)	−0,0044 (1/10)	−0,0019 (3/10)	—

- **A imputação de categorias por floresta é a técnica mais consistente.** Ela foi positiva nos três modelos. A floresta acerta 97% dos `service` escondidos e 99,4% dos `flag`; a moda acerta 57% e 92%.
- **A MLP é a que mais depende da imputação.** Com as duas imputações, ela foi de 0,9623 para 0,9698. Uma árvore consegue isolar um valor imputado errado com um corte; uma rede neural não.
- **O filtro de variância quase zero piorou todos os modelos.** As colunas quase sempre zero, como `num_root` e `root_shell`, são justamente as que acusam os ataques raros nas poucas linhas em que não são zero.
- **Os pares reetiquetados pioraram a random forest** (p = 0,01). O rótulo previsto para eles não é confiável o bastante para virar treino. A limpeza da Etapa 1, que remove os pares, fica como está.
- **Nenhum balanceamento passou** em nenhum modelo com o limiar ajustado. Contando a Etapa 2, foram 4 técnicas: `class_weight`, SMOTE, undersampling e oversampling aleatório. Testes exploratórios de 5 folds com SMOTE+Tomek, SMOTE+ENN, Borderline-SMOTE, ADASYN e Balanced Random Forest também ficaram abaixo da referência. Isso concorda com Elor & Averbuch-Elor (2022) e van den Goorbergh et al. (2022): para classificadores fortes, ajustar o limiar cumpre o papel do balanceamento sem distorcer as probabilidades.

Ajuste de hiperparâmetros

A busca aleatória (Bergstra & Bengio, 2012) melhorou os três modelos. **O maior ganho foi o da MLP (+0,0052).** A random forest, como já se esperava, quase não depende do ajuste. O procedimento tem três passos:

1. sortear os candidatos do espaço de busca, com semente fixa (7);
2. avaliar cada candidato na 1ª repetição (5 folds);
3. confirmar os 3 melhores nos 10 folds. Fica a configuração de maior média nos 10 folds.

Modelo	Candidatos	Espaço de busca	Configuração vencedora	F1-macro antes → depois	Vitórias
MLP (3 sementes)	10	camadas, `alpha`, taxa de aprendizado, lote, paciência	(128, 64), `alpha` = 1e-5, `learning_rate_init` = 0,003, lote 128, paciência 20	0,9698 → **0,9750**	9/10
Random forest	10	`max_features`, `min_samples_leaf`, critério, `class_weight`, `max_depth`	300 árvores, `max_features` = 0,2, `max_depth` = 30, gini	0,9834 → **0,9846**	6/10

Modelo	Candidatos	Espaço de busca	Configuração vencedora	F1-macro antes → depois	Vitórias
Árvore de decisão	25	critério, `max_depth`, `min_samples_leaf`, `ccp_alpha`, `class_weight`, `max_features`	gini, `max_depth` = 16, `ccp_alpha` = 1e-5	0,9664 → **0,9676**	6/10

- **Folhas grandes atrapalham.** Na árvore, `min_samples_leaf` maior que 2 derrubou o F1 para 0,94–0,95. Com 3% de anomalias, uma folha grande "engole" os grupos pequenos de ataques.
- **`class_weight` não ajudou.** Os candidatos com `class_weight="balanced"` ou `"balanced_subsample"` ficaram abaixo dos melhores. É o mesmo resultado do balanceamento.
- **Ganhos pequenos, mas mantidos.** Na árvore e na random forest, o ganho venceu só em 6 de 10 folds. Mesmo assim, ficamos com a configuração de maior média, porque a escolha é entre hiperparâmetros do mesmo modelo, e não entre incluir ou não uma etapa no pipeline.

Versões com `score_ids_legado` e ensembles

Com o `score_ids_legado`, os três modelos sobem, e a random forest continua na frente (0,9909). A média das probabilidades de vários modelos (*soft voting*) não superou a random forest sozinha em nenhuma versão.

O `score_ids_legado` é a saída de um sistema de detecção antigo. Sozinho, ele tem AUC de 0,968 no treino limpo. Ele é legítimo se estiver disponível no momento da detecção, mas a origem dele não é documentada. Por isso, todas as comparações de pré-processamento foram feitas **sem** ele, e as configurações finais foram rodadas de novo **com** ele.

Ensemble (média das probabilidades)	F1-macro	Diferença para a random forest sozinha	Vitórias	p
Sem score: RF + MLP	0,9796	−0,0050	1/10	0,05
Sem score: RF + MLP + árvore	0,9803	−0,0043	0/10	0,02
Sem score: RF + árvore	0,9756	−0,0090	0/10	0,001
Com score: RF + MLP + árvore	0,9911	+0,0002	5/10	0,81

Ensemble (média das probabilidades)	F1-macro	Diferença para a random forest sozinha	Vitórias	p
Com score: RF + MLP	0,9908	−0,0001	5/10	0,92
Com score: RF + árvore	0,9856	−0,0053	0/10	0,005

- **O score vale cerca de +0,006 na random forest** (0,9846 → 0,9909) e também reduz o desvio entre folds pela metade (0,0031 → 0,0016).
- **Sem o score, o ensemble piora a random forest.** A MLP e a árvore erram em conexões em que a floresta acerta, e a média puxa essas conexões para o lado errado.
- **Com o score, o melhor ensemble empata com a random forest** (+0,0002, p = 0,81). Pela regra, ficamos com a random forest sozinha, que é mais simples de explicar e de reproduzir.

Tabela comparativa final

A random forest lidera nas duas versões e em todas as métricas da classe `anomaly`. Todos os valores são médias dos 10 folds da validação 5×2, cada fold com o limiar escolhido fora dele. As métricas por classe vêm da matriz de confusão somada nos 10 folds.

Modelo	Versão	F1-macro	Desvio	F1-macro, limiar 0,5	Limiar mediano	Precisão (anomaly)	Revocação (anomaly)	F1 (anomaly)	F1 (normal)
Random forest	com score	**0,9909**	0,0016	0,9918	0,475	0,9835	0,9813	0,9824	0,9994
MLP	com score	0,9884	0,0027	0,9883	0,430	0,9790	0,9761	0,9775	0,9992
Random forest	sem score	**0,9846**	0,0031	0,9786	0,370	0,9793	0,9612	0,9701	0,9990
Árvore de decisão	com score	0,9793	0,0043	0,9793	0,020	0,9583	0,9618	0,9600	0,9986
MLP	sem score	0,9750	0,0049	0,9750	0,485	0,9741	0,9301	0,9516	0,9984
Árvore de decisão	sem score	0,9676	0,0058	0,9678	0,510	0,9510	0,9241	0,9373	0,9979
Classe majoritária	—	0,4916	0,0000	0,4916	—	0,0000	0,0000	0,0000	0,9832

Na matriz de confusão somada nos 10 folds, a random forest sem score, o modelo de referência da Etapa 4, fica assim:

	Previsto normal	Previsto anomaly
Real normal	106.440	74
Real anomaly	141	3.493

Ela pega **96,1% dos ataques**, e só **2 em cada 100 alarmes são falsos**. Os 74 alarmes falsos equivalem a 0,07% das conexões normais. O ponto fraco são os ataques perdidos: são quase o dobro dos alarmes falsos. O limiar abaixo de 0,5 já compensa parte disso: ele troca um pouco de precisão por revocação.

Decisão, riscos e próximos passos

Decisão: levar a random forest para a Etapa 4, com o pipeline e o limiar abaixo.

- **Pipeline:**
  - `service` / `flag` imputados por uma random forest de 200 árvores;
  - moda no que sobrar e mediana nos números;
  - `log1p` nas contagens e `StandardScaler`;
  - one-hot com os serviços raros juntos;
  - sem balanceamento.
- **Modelo:** 300 árvores, `max_features` = 0,2, `max_depth` = 30.
- **Limiar:** 0,37 sem o score e 0,47 com ele, a mediana dos limiares escolhidos fora de cada fold.

Desempenho esperado no teste: cerca de 0,985 de F1-macro sem o score e 0,991 com ele, desde que o teste tenha a mesma distribuição do treino limpo.

Risco	Por que importa	Como tratamos
O teste pode ter uma distribuição diferente do treino limpo	A validação só estima bem o teste se os dois forem parecidos	Conferido: o teste tem 0 linhas duplicadas e 0 linhas idênticas às do treino, e o score legado se distribui como no treino limpo. Ele já parece deduplicado
O modelo prevê cerca de 4,9% de anomalias no teste, contra 3,3% no treino limpo	O teste pode ter mais ataques, ou tipos de ataque que não estão no treino (Sommer & Paxson, 2010; Engelen et al., 2021)	A Etapa 4 analisa a distribuição das probabilidades no teste

Risco	Por que importa	Como tratamos
Origem não documentada do `score_ids_legado`	Se ele for calculado depois do fato, é vazamento	As duas versões ficam prontas; a decisão fica registrada no relatório final
Ganhos pequenos de hiperparâmetros (6 de 10 folds)	Parte do ganho pode ser ruído da validação	O efeito máximo esperado é de ± 0,001 no F1

Próximos passos para a Etapa 4:

- [ ] Treinar o pipeline no treino limpo inteiro e gerar `submissao_<equipe>.csv` com as checagens de formato.
- [ ] Explicar o modelo: importância por permutação, SHAP e uma árvore substituta.
- [ ] Analisar onde o modelo erra, por `service`, `flag` e faixa do score.
- [ ] Decidir a versão final (com ou sem score) e escrever o relatório de 3 a 5 páginas.

Uso de IA e referências

Uso de IA: o grupo usou o Claude (Anthropic) em três tarefas:

- escrever o código dos experimentos (`experimentos.py` e as células marcadas com `[IA]` no notebook);
- pesquisar a literatura sobre as técnicas testadas;
- revisar o texto deste relatório.

As decisões de modelagem, a regra de comparação e a conferência dos resultados foram do grupo.

Referências

- Arp, D. et al. (2022). *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security.
- Bergstra, J.; Bengio, Y. (2012). *Random Search for Hyper-Parameter Optimization*. JMLR 13.
- Bouckaert, R.; Frank, E. (2004). *Evaluating the Replicability of Significance Tests for Comparing Learning Algorithms*. PAKDD.
- Elor, Y.; Averbuch-Elor, H. (2022). *To SMOTE, or not to SMOTE?* arXiv:2201.08528.
- Engelen, G.; Rimmer, V.; Joosen, W. (2021). *Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study*. IEEE SPW.
- Grinsztajn, L.; Oyallon, E.; Varoquaux, G. (2022). *Why do tree-based models still outperform deep learning on tabular data?* NeurIPS.

- Kadra, A. et al. (2021). *Well-tuned Simple Nets Excel on Tabular Datasets*. NeurIPS.
- Kaufman, S. et al. (2012). *Leakage in Data Mining: Formulation, Detection, and Avoidance*. ACM TKDD.
- Kuhn, M.; Johnson, K. (2019). *Feature Engineering and Selection*. CRC Press.
- Lipton, Z.; Elkan, C.; Narayanaswamy, B. (2014). *Optimal Thresholding of Classifiers to Maximize F1 Measure*. ECML PKDD.
- Nadeau, C.; Bengio, Y. (2003). *Inference for the Generalization Error*. Machine Learning 52.
- Northcutt, C.; Jiang, L.; Chuang, I. (2021). *Confident Learning: Estimating Uncertainty in Dataset Labels*. JAIR 70.
- Sommer, R.; Paxson, V. (2010). *Outside the Closed World: On Using Machine Learning for Network Intrusion Detection*. IEEE S&P.
- Stekhoven, D.; Bühlmann, P. (2012). *MissForest: non-parametric missing value imputation for mixed-type data*. Bioinformatics 28.
- Tavallaee, M. et al. (2009). *A Detailed Analysis of the KDD CUP 99 Data Set*. IEEE CISDA.
- van den Goorbergh, R. et al. (2022). *The harm of class imbalance corrections for risk prediction models*. JAMIA 29.
- Zheng, A.; Casari, A. (2018). *Feature Engineering for Machine Learning*. O'Reilly.